# Arrays, Hashing and Two Pointers

**Topic 2 of 10**

**38 questions | skeleton and writing space only**

**Each question: the prompt, the function or class signature, and ruled lines to write your own solution.**

## 1. Maximum-Area Rectangle from Planar Points

**HARD | asked 2026-09-29 | math, hashmap**

Given 2D points, find the maximum area of a rectangle whose four corners are input points; sides need not be parallel to the axes. Input: a list of 2D coordinate points. Return the maximum area of any rectangle whose four vertices are all input points. Rectangle sides do not need to be parallel to the coordinate axes.

```
def max_rect_area(points):
```

## 2. Smallest Missing Positive Integer in Linear Time and Constant Extra Space

**HARD | asked 2026-09-09 |**

You are given an unsorted list of integers nums, which may contain negative numbers, zeros, duplicates, and values far larger than the length of the list. Return the smallest positive integer, that is the smallest integer that is at least 1, that does not appear in nums. Aim for that tighter target. ### Function Signature python def first_missing_positive(nums: list[int]) -> int: ### Rules - Target complexity: O(n) time and O(1) auxiliary space, where n = len(nums).

```
def first_missing_positive(nums):
```

## 3. Necklace Cut into Two Halves with Equal D/R Counts

**HARD | asked 2026-05-03 | string, two-pointer, math-reasoning**

**Examples: s = "DRDR" -> cut between index 1 and 2: "DR" + "DR". s = "DDRR" -> 1-cut not balanced ("DD" vs "RR"); rotate, two cuts at offsets 1 and 3: pieces "DR" and "DR".**

```
def necklace_two_cuts(s):
```

## 4. Find longest duplicated substring

**HARD | asked 2025-12-08 |**

**You are given a string s consisting of lowercase English letters. A **substring** of s is any contiguous sequence of characters, i.e., s[i..j) for 0 <= i < j <= len(s). Design an algorithm to find **any one** of the longest substrings that appears **at least twice** in s. The two occurrences may overlap. If no substring appears at least twice, return the empty string. ### Requirements - Input: a string s of length n (e.g., 1 <= n <= 2 x 10^5).**

```
def longest_dup(s):
```

## 5. Connected Crop Arrangement on a Grid

**MEDIUM | asked 2026-10-04 | grid, array, greedy**

Given an N x M grid and K crop counts summing to N*M, fill every cell so each crop's cells form one 4-connected region; return any valid arrangement. A garden is an N x M grid. There are K crop types labeled 1..K; crop i must occupy exactly count[i] cells. Guaranteed: sum(count) = N * M, so every cell is used. For every crop type, all of its cells must form one connected component under 4-directional adjacency (up, down, left, right; diagonals do not count).

```
def arrange_crops(n, m, counts):
```

## 6. Range Duplicate Check with Constant-Time Queries

**MEDIUM | asked 2026-09-29 | array, sliding-window, hashmap, prefix-sum**

Decide whether the elements in a range of an array are all unique. Progression: set-based duplicate check -> sliding window -> many range queries where reads dominate, requiring O(1) per query after linear-time preprocessing / updates with linear space. Base: given an array and a range [l, r], determine whether the elements in that range are all unique. The problem is staged: Easy: duplicate detection with a set. Sliding-window duplicate checking.

```
class RangeUnique:

    def __init__(self, a):
```

```
    def rebuild(self):
```

```
    def unique(self, l, r):
```

```
    def update(self, i, value):
```

## 7. Temperature Monitoring Data Structure

**MEDIUM | asked 2026-09-10 | data-structure, streaming, sliding-window**

Implement temperature insertion, a moving average, and a constant-time maximum query for a recent window. Clarify whether the window is based on reading count or elapsed days.

```
class Monitor:

    def __init__(self, n):
```

```
    def insert(self, x):
```

## 8. Longest Same-Color Grid Sequence

**MEDIUM | asked 2026-09-08 | array, grid**

Given a grid of colors, return the longest consecutive same-color sequence along a horizontal, vertical, or diagonal line. Input is a two-dimensional grid of colors such as R, G, and B. Return the length of the longest consecutive sequence of a single color horizontally, vertically, or diagonally. Examples: R R R G R R B B G R R R B G B R Answer: 4

```
def longest_same_color_line(g):
```

## 9. Next-Word Predictor

**MEDIUM | asked 2026-09-02 | string, hashmap, data-structure**

**Process training data and predict the most frequent next word for a supplied word. Maintain prediction state during processing so prediction can run in constant time.**

```
class Predictor:

    def __init__(self, orders=(2, 1)):

    def train(self, sentence):

    def update(self, k, ctx, nxt):

    def predict(self, *context):

    def top_k(self, word, k):
```

## 10. Piano Hand Movement Grouping

**MEDIUM | asked 2026-07-14 | array, simulation, sliding-window**

**Examples: With notes = [1, 3, 5, 7, 9, 11] and k = 4, the stated output is 2. Earlier examples use [1,2,3,4,5] -> 0 and [5,9,1] -> 1, without stating the reach parameter.**

```
def groups(notes, k):
```

## 11. Filter Repeated Robot Status Messages

**MEDIUM | asked 2026-07-05 |**

**Overview: Filter a timestamped robot status stream so a message reappears only when strictly more than ten seconds have passed since its last displayed occurrence. Preserve input order, track each exact case-sensitive message independently, and ensure hidden repeats do not reset the interval.**

```
def filter_status(stream, gap=10):
```

## 12. Website Activity Analytics: Unique Users + Session Time

**MEDIUM | asked 2026-06-26 | python, hashmap, data-analysis, sql**

**Given website action records with user_id, timestamp, and activity, compute per-activity unique-user counts and average time spent on the site per user in seconds. The screen paired this Python task with a lighter SQL section and emphasized edge-case discussion over perfect final code. Input is a collection of user-activity records.**

```
def unique_users_by_activity(records):
```

## 13. Find a Horizontal Cut That Bisects Rectangle Area

**MEDIUM | asked 2026-06-04 |**

**Overview: Solve a computational-geometry problem that asks for the horizontal line dividing the combined area of many disjoint rectangles in half. The exercise emphasizes numerical precision, large coordinate ranges, performance at 200,000 rectangles, and careful treatment of partially intersected shapes.**

```
def cut_height(rects, share=Fraction(1, 2)):
```

## 14. Gold Chain Split into Two Equal Halves

**MEDIUM | asked 2026-04-21 | array, hashmap**

**Onsite coding interview 1: given a gold chain represented by an array of segment weights, remove exactly one segment and check whether the remaining segments -- when reconnected -- can be cut once more to give two halves of equal total weight.**

```
def gold_chain(a):
```

## 15. Solve several coding interview problems

**MEDIUM | asked 2026-03-04 |**

**Overview: This set of problems evaluates proficiency in string processing and combinatorial parsing, arbitrary-precision arithmetic using string representations, pointer/ancestor-chain intersection detection, and computational geometry for overlap counting, emphasizing algorithmic design, correctness, and performance.**

```
def add_strings(a, b):
```

## 16. Solve array-sum and city-community problems

**MEDIUM | asked 2026-02-20 |**

**Overview: This question evaluates array-processing and grid/graph connectivity skills, covering counting subarrays with a target sum and identifying connected building communities in a grid; it belongs to the Coding & Algorithms domain and involves streaming/online algorithm reasoning as well as grid traversal concepts.**

```
def count_subarrays(nums, k):
```

## 17. Check pivot after removing one element

**MEDIUM | asked 2026-02-12 |**

**Overview: This question evaluates array-manipulation skills, cumulative-sum/prefix-sum reasoning, and robustness in handling negative values and duplicates, testing algorithmic problem-solving competency in the Coding & Algorithms domain and focusing on practical application rather than purely conceptual understanding.**

```
def pivot_after_removal(a):
```

## 18. Min deletions to avoid overlap in first k

**MEDIUM | asked 2026-02-12 |**

**Overview: This question evaluates array and sequence manipulation skills, set-based reasoning about value overlap, and the ability to compute minimal edits while preserving relative order constraints.**

```
def min_deletions(a, k):
```

## 19. Sum of Good Subarrays (Adjacent Diff +/-1)

**MEDIUM | asked 2026-02-08 | array, two-pointer**

**MLE L5 coding round: sum over all contiguous subarrays whose elements form a strictly monotonic +/-1 step sequence (every adjacent pair differs by exactly +1, OR every adjacent pair differs by exactly -1). Required time complexity is O(N). Input: integer array arr of length N. A subarray is good if its consecutive differences are either all +1 (monotone increasing by 1) or all -1 (monotone decreasing by 1). Singleton subarrays are always good.**

```
def good_subarray_sum(a):
```

## 20. Run-Length Vector Storage + Dot Product

**MEDIUM | asked 2026-01-21 | data-structure, two-pointer**

**Examples: Logical A = [3, 3, 3, 0, 0, 0, 4, 4] Logical B = [1, 1, 1, 1, 2, 2, 2, 2] Encoded A (RLE): [(3,3),(0,3),(4,2)] Encoded B (RLE): [(1,4),(2,4)] Dot product = 3·1·3 + 3·1·0 + 0·1·0 + 0·2·0 + 4·2·2 = 9 + 0 + 16 = 25**

```
def rle_dot(a, b):
```

## 21. Reconstruct original array from doubled shuffle

**MEDIUM | asked 2026-01-04 |**

**Overview: This question evaluates array manipulation, multiset/frequency reasoning, pairing logic, and careful handling of edge cases such as zeros and negative numbers.**

```
def find_original(changed):
```

## 22. Maximize 1D deliveries within distance and minimize total distance

**MEDIUM | asked 2025-12-27 |**

**Overview: This question evaluates combinatorial optimization and constrained matching skills, focusing on reasoning about distance-based pairings, lexicographic objective prioritization, and edge-case handling such as duplicates and empty inputs.**

```
def deliveries(orders, drivers, D):
```

## 23. Two-Day Log Intersection under Memory Limit

**MEDIUM | asked 2025-11-27 | hashmap, two-pointer, streaming**

**PhD intern coding round 1: two log files with (timestamp, obj_id, client_id) entries. Find obj_ids that (a) appear in both files and (b) have at least two distinct client_ids. Input: two log files; each line is (timestamp, obj_id, client_id). Find every obj_id such that: (a) appears in both files, and (b) is associated with at least 2 distinct client_ids (aggregated across both files).**

```
def log_objects(day1, day2):
```

## 24. Add two big integers from digit lists

**MEDIUM | asked 2025-11-24 |**

**You are given two non-empty singly linked lists (or arrays) representing two non-negative integers. The digits are stored in **reverse order**, and each node contains a single digit from 0 to 9. Your task is to add the two numbers and return the sum as a new linked list, also in reverse order. You may assume: - Each list has at least one node. - The integers do not contain leading zeros, except the number 0 itself.**

```
def add_digits(a, b):
```

## 25. Minimize calls to find all bad test pairs

**MEDIUM | asked 2025-11-05 |**

You have \(n\) atomic tests \(t_1, t_2, \dots, t_n\). You are given access to a black-box function: python bool runTest(set<Test> S) which behaves as follows: - runTest(S) == True if the subset \(S\) contains **no** incompatible (bad) pair of tests. - runTest(S) == False if the subset \(S\) contains **at least one** incompatible (bad) pair of tests. There is some unknown set of unordered **bad pairs** \((t_i, t_j)\), where \(i\ne j\).

```
def find_bad_pairs(tests, run_test):
```

## 26. Rotate Any mxn 2D Matrix

**MEDIUM | asked n/a | array, grid**

Examples: [[1,2,3],[4,5,6]] (2x3) rotated 90deg CW -> [[4,1],[5,2],[6,3]] (3x2).

```
def rotate_cw(matrix):
```

## 27. Find the minimum window substring.

**MEDIUM | asked n/a |**

**Find the minimum window substring.**

```
def min_window(s, t):
```

## 28. Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.

**MEDIUM | asked n/a |**

**Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.**

```
def smallest_e(kids, sellers):
```

### 29. Shortest Subarray with K Distinct Integers

**MEDIUM | asked n/a |**

**Given an array nums and an integer k, find the length of the shortest contiguous subarray that contains at least k distinct integers. If no such subarray exists, return -1.**

```
def shortest_k_distinct(nums, k):
```

### 30. Sum of Two Squares in Two Ways

**MEDIUM | asked n/a |**

**Given an integer n, find all integers s < n such that s can be expressed as the sum of two squares in exactly two distinct unordered ways. For example: 50 = 1^2 + 7^2 = 5^2 + 5^2.**

```
def sums_in_k_ways(n, k=2, zero=False):
```

## 31. Robot Status Message Deduplication Window

**EASY | asked 2026-09-10 | hashmap, simulation**

**Process timestamped robot messages and display a message only when it has not been displayed within the prior 10 seconds. Hidden arrivals do not refresh the last-displayed timestamp, and the exact 10-second boundary must be clarified. Process a stream of (time, message) status events from a robot. If the same message was displayed within the past 10 seconds, hide the new event. Otherwise display it and update that message's last-displayed time.**

```
class Dedup:

    def __init__(self, window=10, boundary_allo
      wed=True):
```

```
    def too_soon(self, gap):
```

```
    def show(self, t, msg):
```

## 32. Reverse a linked list.

**EASY | asked n/a |**

**Given the head of a singly linked list, reverse the list and return its head. Implement a function reverseList that takes the head of the linked list as input and returns the new head of the reversed list. Refer to the ListNode class/struct and do not comment it out.**

```
def reverse_list(head):
```

## 33. Sort a binary array in linear time.

**EASY | asked n/a |**

**Sort a binary array in linear time.**

```
def sort_binary(arr):
```

## 34. Is this a valid palindrome?

**EASY | asked n/a |**

**Given a string s, return true if it is a palindrome, and false otherwise. A string is considered a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Write a function isPalindrome that implements this logic.**

```
def is_palindrome(s):
```

## 35. Find the Duplicates

**EASY | asked n/a |**

**Given two sorted arrays arr1 and arr2 of passport numbers, implement a function findDuplicates that returns an array of all passport numbers that are both arrays. Note that the output array should be sorted in ascending order. Let N and M be the lengths of arr1 and arr2, respectively. Solve for two cases and analyze the time & space complexities of your solutions: M ~ N - the array lengths are approximately the same M >> N - arr2 is much bigger than arr1. Try two nested loops.**

```
def common(a, b):
```

## 36. Reverse a Sentence

**EASY | asked n/a |**

**You are given an array of characters arrthat consists of sequences of characters separated by space characters. Each space-delimited sequence of characters defines a word. Implement a function reverseWords that reverses the order of the words in the array in the most efficient manner.**

```
def reverse_words(arr):
```

## 37. Determine if two words are sorted in lexicographic order.

**EASY | asked n/a |**

Determine if two words are sorted in lexicographic order.

```
def in_order(a, b, rank):
```

## 38. Parity Alternating Subarray Query

**EASY | asked n/a |**

Given an array nums and multiple queries, each query asks whether a subarray nums[l:r] satisfies the condition that adjacent elements alternate in parity (odd-even-odd-even or even-odd-even-odd). Preprocess the array to answer each query in O(1) time.

```
class ParityQueries:

    def __init__(self, nums):
```

```
    def alternates(self, l, r):
```